The College Scorecard Dataset

How the data is actually structured - a plain-English orientation before you start EDA

Generated 20 September 2026 from `data/CollegeScorecardDataDictionary.xlsx` (unmodified) and the live College Scorecard API. Group Project I - The College Scorecard Dataset.

3,308	178	30	6,273
institution-level variables	field-of-study variables	annual datafiles, 1996-97 to 2025-26	institutions in the API

The 60-second version

1. **It is not one table.** There are two separate datasets that share a key: **institution-level** (one row per college per year) and **field-of-study-level** (one row per college x CIP program per year).
2. **It is wide, not long.** Institution-level data has roughly **3,308 columns** in the API and ~3,000+ in the annual CSV files, for ~6,300 rows.
3. **Every column has three names.** The raw CSV name (`ADM_RATE`), the dotted API path (`latest.admissions.admission_rate.overall`), and a human description. Only the dotted path works in the API; only the raw name works in the CSV files. Confusing these two is the #1 source of wasted hours.
4. **Time is attached to the column, not the row.** The API stacks all years into the column name: `2019.student.size` . `latest.*` means "most recent value we have for this specific metric" - and that is *not* the same year for every metric.
5. **Missing is normal and comes in flavours.** Blank, `null` , privacy-suppressed, and not-applicable-to-this-school all look similar downstream. Treating them all as plain `NA` is fine to start, but it is a real data-quality finding in its own right.
6. **About 4 in 10 variables are coded integers.** A `1` in `school.ownership` means "Public", not the number 1. The workbook's continuation rows hold those decodings; they are flattened for you in `reference/05_value_labels.csv` .
7. **The two datasets have different shapes.** Institution variables are flat (one path = one number). Field-of-study variables are keys inside one nested array per institution, so they are unnested rather than selected. See the end of section 1.

1. There are two datasets, not one

Everything in the Scorecard belongs to exactly one of these. Deciding which one you need is the first decision of the project.

	Institution-level	Field-of-study-level
Grain	One row per institution per year	One row per institution x CIP program per year
What it answers	How does this college do overall?	How does *this major at this college* do?
API fields	3,308 (`<year>.<category>.<metric>`)	178 (`<year>.programs.cip_4_digit.*`)

	Institution-level	Field-of-study-level
CSV files	`MERGED_YYYY_YY_PP.csv` plus a `Most-Recent-Cohort` file	`FieldOfStudyDataYYYY_YY.csv` plus a most-recent file
Years available	30 files, 1996-97 through 2025-26	9 files, 2014-15 through the most recent
Workbook tabs	`Institution_Data_Dictionary`, `Institution_Cohort_Map`, `Most_Recent_Inst_Cohort_Map`	`FieldOfStudy_Data_Dictionary`, `FieldOfStudy_Cohort_Map`

Practical consequence

If your question is about *access, affordability or overall student outcomes*, use the institution-level data. The field-of-study data only starts in 2014-15, so a field-of-study time series over the full history is impossible. See the assignment's Goal 4: outcomes can cover access, affordability, student outcomes "or other categories" - you get to pick, and the pick is largely a data-availability question.

The one shape difference you must know about

The institution data is **flat**: one field path is one number. The field-of-study data is **nested**. Each institution is a single object, and inside it there is an array - one element per CIP program - and the 178 field-of-study dictionary variables are the keys *inside* those elements:

```
latest.programs.cip_4_digit   -->  [                    <-- array, one element per program
    { unit_id: 100654, code: '01.0101', title: 'Agricultural Business...',
      credential: { level: 3, title: 'Bachelor Degree' },
      counts:  { ipeds_awards1: ..., ipeds_awards2: ... },
      debt: {...}, earnings: {...}, repayment: {...}, distance: ... },
    { ...next program... }
]
```

So `latest.programs.cip_4_digit[].earnings.4_yr.overall_median_earnings` is **not** something you can put in `fields=`. You request the array and then unnest it. In R that is `tidyr::unnest_longer()` or `jsonlite::fromJSON(..., simplifyVector = TRUE)`; in pandas it is `json_normalize(record_to_explode = 'latest.programs.cip_4_digit')`. The exact key list is enumerated in `reference/07_field_of_study_variables.csv`.

API quirk you will hit in the first five minutes

When you request that array, the JSON response does **not** nest it. It comes back under a single key whose name is the literal dotted string `latest.programs.cip_4_digit`, as a sibling of `id` - not under `latest` > `programs`. So `result$latest$programs` is `NULL` and it looks like there is no data. Read `result[['latest.programs.cip_4_digit']]` instead.

Why this matters for your EDA

If you unnest the array, your unit of analysis silently changes from *institution* to *institution x program*. Row counts explode, one school contributes dozens of rows, and any average you compute is now weighted by how many

programs a school happens to offer. Aggregate back to institution level with an explicit weighting decision, or state clearly that the analysis is at program level.

2. Three ways to get the same numbers

Route	How	Use it when
Annual CSV files	`MERGED_1996_97_PP.csv` … `MERGED_2025_26_PP.csv`, plus `Most-Recent-Cohort-*.csv`	You want the full 3,000+ column firehose, or an offline reproducible snapshot. One file per year means 30 imports and a lot of RAM.
REST API (direct)	`GET https://api.data.gov/ed/collegescorecard/v1/schools?api_key=...&fields=...`	You want a subset of columns, one year, or a live/refreshable pull. Max `per_page=100`, rate limit 1,000 requests/hour.
`rscorecard` in R	`sc_init() |> sc_filter(...) |> sc_select(...) |> sc_get()`	It is the same API with R-friendly verbs and automatic pagination. This is what the project's `APIWalkthrough.md` sets up.

The API is the API. `rscorecard` does not have its own copy of the data - it calls the same URL and returns a tibble. So whatever you learn about field paths and missingness applies either way.

3. The variable namespace: 10 categories

A working API field path is a dotted path with a time prefix:

```
<time> . <category> . <sub-topic> . <metric>
   2019  . admissions . admission_rate     . overall
  latest  . cost       . net_price         . consumer.by_income_level.0-30000
```

Prefix	Category	Variables	Coded	What it covers
`id, location.*`	root	6	0	Identifiers for the institution itself (Unit ID, OPE ID, lat/lon, school code). These are the join keys. Not year-specific.
`<year>.school.*`	school	50	29	Who the school is: name, address, sector/ownership, Carnegie classification, accreditor, minority-serving flags, Title IV status, endowment, faculty.
`<year>.admissions.*`	admissions	32	1	How hard it is to get in: admission rate and SAT/ACT score distributions.
`<year>.cost.*`	cost	85	0	What it costs: tuition and fees, average net price overall and by family-income band.
`<year>.aid.*`	aid	111	0	What students get and what they owe: Pell/loan take-up rates, median debt at graduation and for non-completers, PLUS loans.
`<year>.student.*`	student	131	0	Who is enrolled and how many: total size, race/ethnicity shares, first-generation share, family income, full-time/part-time split.
`<year>.academics.*`	academics	247	0	What is taught: share of degrees by CIP field, programs offered, average months to completion by program.
`<year>.completion.*`	completion	1,367	0	Whether students finish: 100%/150%/200% completion and outcome rates, broken down by Pell status, loan status, first-gen status, race and time since entry. The single largest category.
`<year>.earnings.*`	earnings	185	0	What graduates earn: median/mean earnings and counts of working vs not working 1-11 years after entry or after completion, plus earnings thresholds.
`<year>.repayment.*`	repayment	1,094	0	Whether loans get repaid: cohort default rates, repayment rates at 1/3/5/7 years, and balance-based repayment rates (BBRR/DBRR) split by demographics and cohorts.

Two exceptions to remember

- The identifiers are **not** inside a category. Use `id`, `ope8_id`, `ope6_id`, `location.lat`, `location.lon` - never `root.id`.
- Scores of variables are **variants of the same metric**. For example `completion.completion_rate_4yr_150nt` (individual cohort) vs `completion.completion_rate_4yr_150nt_pooled` (two years pooled to reduce volatility). Read the trailing word before you trust a number.

4. One number, three names - the thing that wastes the most time

The dictionary gives you **both** naming systems side by side. Here is what the gap looks like for real variables:

API path (use in the API / rscorecard)	Raw column (use in the CSV files)	What it means	Type	Years	Coded?
`latest.admissions.admission_rate.overall`	`ADM_RATE`	Admission rate	float	24/30	no
`latest.cost.net_price.public.by_income_level.0-30000`	`NPT41_PUB`	Average net price for $0-$30,000 family income (public institutions)	integer	16/30	no
`latest.aid.median_debt.completers.overall`	`GRAD_DEBT_MDN`	The median debt for students who have completed	float	24/30	no
`latest.student.size`	`UGDS`	Enrollment of undergraduate certificate/degree-seeking students	integer	28/30	no
`latest.completion.completion_rate_4yr_150nt`	`C150_4`	Completion rate for first-time, full-time students at four-year institution...	float	28/30	no
`latest.earnings.10_yrs_after_entry.median`	`MD_EARN_WNE_P10`	Median earnings of students working and not enrolled 10 years after entry	integer	7/30	no
`latest.repayment.3_yr_default_rate`	`CDR3`	Three-year cohort default rate	float	14/30	no
`latest.school.ownership`	`CONTROL`	Control of institution (IPEDS)	integer	30/30	yes

Rule of thumb

- If it is **lowercase with dots**, it is an API path. Pattern: `latest.category.metric`.
- If it is **SHOUTY_UPPER_SNAKE**, it is a raw CSV column. Pattern: `ADM_RATE`, `C150_4`, `MD_EARN_WNE_P10`.
- The cohort-map tabs are keyed on the **raw** name. The API only knows the **dotted** name. `reference/03_institution_variables.csv` holds both in one row so you can translate either way.

5. Time lives in the column name, and it is ragged

This is the part that surprises everyone. In the API there is no `year` column - the year is baked into the field name:

```
latest.student.size      -> most recent value available for THIS metric
2019.student.size        -> the value as it stood in the 2019-20 datafile
2019.cost.tuition.in_state
2019.earnings.10_yrs_after_entry.median
```

Question	Where the answer is
Which years exist for a given variable?	`Institution_Cohort_Map` - variable rows x 30 year columns. A cell of `NA` means "this metric does not exist in that datafile".
What exactly is being measured in that year?	Same tab - the cell text names the cohort, e.g. "AcadYr 2019-20, reported in IPEDS DCY2019-20 *-or-* NSLDS pooled AY2017-18, AY2018-19 cohort". The `-or-` means the metric has two different sources/definitions across the years.
Why is `latest.*` a different year in every column?	`Most_Recent_Inst_Cohort_Map` - one row per variable naming the cohort it uses in the most-recent file. The API documentation says this explicitly: data are published at different times during the year.
Is a variable available for all 30 files?	`reference/06_cohort_coverage.csv` - a single row per variable with `years_with_data_count`, `first_year_with_data`, `last_year_with_data`.

The shape of the coverage, in numbers

Of the 3,308 institution variables:

- **247** have a value somewhere in all 30 annual files (mostly identifiers and slow-moving school characteristics).
- **3,026** exist in some years only - the classic case, and the reason a naive `read.csv` of every year will not line up column-wise.
- **31** are coded `NA` in every one of the 30 files. They still exist in the API. Do not build a time series on these without checking first.

Caveat: a low count is not always absence. Identifiers and slow-moving fields (`LATITUDE`, `ACCREDAGENCY`, `CITY`) show `1` because the workbook only documents them against the most recent datafile. The cohort map records what the publisher bothered to document, not always what the file contains.

6. What a cell can actually contain

Situation	What you see	What to do
Ordinarily missing	`null` in JSON, empty in the CSV	R reads it as `NA`. Count it; it is usually the largest "value" in the column.
Privacy suppressed	Historically the literal string `PrivacySuppressed` in the datafiles. The API's job is to give you `null` instead.	Keep it distinct from ordinary missing if you can - suppression tells you the cell is small and non-zero, which is information. Watch for variables whose name contains `_supp` / `suppressed`.
Not applicable to this school	`null`. E.g. an admission rate for an open-admissions or graduate-only school.	This is structural, not random. Missingness patterns are themselves a finding - the assignment explicitly invites you to look at relationships in missingness.
Coded / factored	An integer whose meaning is a lookup. **30 of 3,308** variables have this, covering **277** value/label pairs.	Decode before summarising. All decodings are flattened into `reference/05_value_labels.csv`.
Pooled / moving average	A separate column with `_pooled`, `_PP`, `_POOLED_SUPP`, `POOLYRS` in the name.	It is a two-year average, not a single year. Never mix pooled and unpooled in the same time series.
Denominator, not a rate	Names like `completion_cohort_*`, `*_DENOM`, `COUNT_*`, `outcome_cohort*`.	A rate with a tiny denominator is noise. Always carry the count column next to the rate column so you can filter on it.

7. The nine things that will bite you

1. **Raw name vs API path.** `ADM_RATE` is not a valid API field; `admissions.admission_rate.overall` is not a CSV column. (See section 4.)
2. `latest` **is not a year.** It is a per-column "newest available" rule. Cross-sectional analysis across different `latest` columns mixes datafiles.
3. **3,000+ columns will not fit comfortably in memory or on a plot.** Select by category first, then by cohort coverage.
4. **~2,200 of the variables come from NSLDS, not IPEDS.** Loan and earnings variables have very different coverage and lags from enrolment variables. The `SOURCE` column in `reference/03_institution_variables.csv` tells you which.
5. **Suppressed and pooled variants are separate columns.** A blank in the main rate column plus a value in the `*_pooled` column is a normal combination, not a bug.
6. **OPE ID is not Unit ID.** `ope8_id` / `ope6_id` identify the OPE institution and can group multiple campuses. Aggregations that use the wrong key silently double-count or drop campuses.
7. **Rows are institutions, and the institution list changes over time.** Schools open, close and merge. 6,273 records exist today; a 1997 file has far fewer. Any year-over-year join leaks missingness.
8. **The dictionary is dated and versioned.** The workbook in `data/` is the June 2026 release. If you pull from the API later, new columns may appear that are not in this dictionary. Check `reference/02_changelog.csv`.
9. **Field-of-study variables are nested, not flat.** `latest.programs.cip_4_digit[].*` paths cannot be passed to `fields=`; you request the array and unnest it, which changes your unit of analysis. (See section 1.)

8. A concrete first EDA session

The goal of session one is *not* to find a finding. It is to answer "what do I actually have?" in about an hour.

Step 1 - pull a small, named sample through the API

```
library(rscorecard)

sc_init() |>
  sc_filter(stabbr == "FL") |>
  sc_select(id, school.name, school.state, school.ownership, latest.student.size,
              latest.admissions.admission_rate.overall, latest.cost.tuition.in_state,
              latest.cost.net_price.consumer.by_income_level.0-30000,
              latest.aid.pell_grant_rate, latest.aid.median_debt.completers.overall,
              latest.completion.completion_rate_4yr_150nt,
              latest.earnings.10_yrs_after_entry.median,
              latest.repayment.3_yr_default_rate) |>
  sc_get() |>
  as_tibble()
```

Every path in that block is in `reference/08_api_field_paths.txt`. Keep `per_page` modest; do not try to pull 3,000 fields at once.

Step 2 - measure the shape before you plot anything

```
dim(df)
names(df)
sapply(df, class)

# how much is missing, per column, worst first
sort(colMeans(is.na(df)), decreasing = TRUE) |> head(30)
```

That `sort(colMeans(is.na(df)))` line is the single highest-value command in the whole project. Compare its output against the `years_with_data_count` column in `reference/06_cohort_coverage.csv` - if a variable is missing 90% of the time in your sample but has 30/30 years of coverage, your sample is the problem, not the data.

Step 3 - pick the universe on purpose

```
# 6,273 records exist; 6,243 report that they are operating.
# Most published analyses also restrict to Title IV, degree-granting, US-only:
sc_init() |>
  sc_filter(school.operating == 1,
            school.degrees_awarded.predominant != 0) |>
  sc_select(id) |> sc_get() |> nrow()
```

Write down the number you get and put it in your report. Every later count is relative to this decision, and it is the kind of choice a grader will ask about.

Step 4 - decode before you describe

```
# WRONG: this looks like a numeric summary
summary(df$school.ownership)

# RIGHT: join the lookup, then summarise
labels <- readr::read_csv("Docs/DataReference/reference/05_value_labels.csv")
labels |> dplyr::filter(variable_name == "CONTROL")
```

Step 5 - then, and only then, do the usual

- Distribution of each continuous variable you selected (`hist` / `boxplot`), with the count column beside it.
- Missingness map - `naniar::vis_miss()` or `VIM::aggr()` - on a subset of ~40 columns.
- Correlation matrix across the numeric subset, to find clusters of near-duplicate columns.
- A data-quality score per column, using the completeness and free-of-error formulas from the assignment.

9. Where to look things up

You want to know...	Open this
Every variable in one flat table, with both names	`reference/03_institution_variables.csv`
The same thing, readable in a browser or on GitHub	`reference/04_institution_variables_by_category.md`
Which API fields to paste into `sc_select()`	`reference/08_api_field_paths.txt`
What a code like `ownership = 1` means	`reference/05_value_labels.csv`
Whether a variable exists in 1997 or 2023	`reference/06_cohort_coverage.csv`
What an abbreviation in the cohort map means	`reference/01_glossary.csv`
When a variable was added, renamed or re-typed	`reference/02_changelog.csv`
Whether these API paths still work	`reference/09_api_path_validation.csv`
What the eight workbook tabs are	`reference/00_readme_and_key_tabs.md`
The long-form statistical methodology	The two PDFs linked in the workbook README (`InstitutionDataDocumentation.pdf` and `FieldOfStudyDataDocumentation.pdf`)

Live-API check

Every API field path printed in this document was checked against the live API on 20 September 2026: **3,486 of 3,486** came back from the service. For the flat institution fields, **3,308 of 3,308** were echoed back by the API. The 178 field-of-study keys were checked by pulling the `latest.programs.cip_4_digit` array and confirming the key

exists inside an array element: **178 of 178** confirmed. These are *not* valid `fields=` values - see section 1. Full result, with the status of every single path, in `reference/09_api_path_validation.csv`.

Built by `scripts/build_data_reference.py` on 20 September 2026. Source workbook `data/CollegeScorecardDataDictionary.xlsx` was read only - nothing in this pack modifies the original project files. Counts reflect a total of 3,308 institution variables and 178 field-of-study variables.